

CSC435 Programming Assignment - III

Chameleon resource used: compute_cascadelake_r

1. Datastructure used:

a. Map.

i. There are two structures in the map data structure. I have used both of them.

ii. First I have used `unordered_map` to store the word and it's associating data.

```
(std::unordered_map<std::string, IndexingData> indexMap;)
```

iii. Second I have used `map` to store the frequency count in (descending order) and the filepath associate with the word.

```
(struct IndexingData  
{  
    // Use map sorted by frequency in descending order  
    std::map<unsigned int, std::string, std::greater<unsigned int>>  
    list;
```

```
    ...  
};)
```

iv. I have used these data structures for two reasons, `unordered_map` helps to retrieve the data in $O(1)$ (very efficient retrieval) and `map` helps with reordered storage of data and also faster retrieval.

2. Creation of global index(in safe way):

a. Mutex.

i. I have used `std::mutex mtx;` in the `IndexStore` class.

ii. When inserting global index/updating the global index, I look the class object using mutex which makes sure R/W operations are done by only that specific thread which has locked the class.

Continue to Next Page

CSC435 Programming Assignment - III

3. Indexing and search performance:

- a. How is the performance influenced?
 - i. As the dataset size increases the need of memory and computation increases, which is a direct factor affecting indexing and search(in some cases) performance.
 - ii. Check the plot diagrams in the end, how indexing and search is affected based on dataset size.
- b. Performance exhibition from the solution:
 - i. The performance of indexing and search were very much related with each other's run-time.
 - ii. As it took so much time for indexing, most of the operations were performed during this stage.
 - iii. As the dataset size was huge the above stage took so much time for loading, indexing, organizing and storing the data ready for the search operations (which needs to be as fast as possible, instantaneous in some cases).
 - iv. Searching was fast because the data is organized in way where the elements are retrieved very quickly.

4. Challenges and Conclusion:

- a. Things which could have done better.
 - i. Regarding the data structures, if I would have stored data more efficiently, may be the performance of indexing could have been much better.
- b. Major Challenges faced.
 - i. One of other major challenge was how to store the indexing data?
 - 1. Do I store it in a vector of map? Do I store it in a map of vectors? Do I store it in an array? Do I store it in a Tree?
 - 2. If I'm sorting them in any container do I need to maintain their order?
 - 3. In case if the order is not sorted how do I perform the search? Do I just sort it before the search?

CSC435 Programming Assignment - III

- ii. Issues related to server - client communication and multi-threading.
 - 1. It took some time for me to figure out how to safely communication between the server and the client.
 - 2. I have spent a lot of time debugging the code in the prints as I was not able to run the code in windows because the code was using Linux/Ubuntu based libraries.
 - 3. There was one bug where I had Literally spent 14 hours of time debugging a deadlock. But in the end the fix was just to make one specific thread from join to detach.
 - 4. I also wanted to optimize the local and networked operations, so I followed the below procedure.
 - a. First I made sure I handled server – client communication correctly without leaving any edge cases.
 - b. As I already had my index and search implemented, I synced that code and test it to make sure it worked.
 - c. Then I created a message to send client id to all clients, they append their client id with the file name at the time of sending to server, this helped me reduce redundant operations.
 - d. I also instantly sent success communication when server received data, this way I can make sure server does its work at the same time client can send the prepare for next batch queries.
 - e. Additional thing which I wanted to do was to create local threads for each client and help them with the indexing.

CSC435 Programming Assignment - III

- f. One other thing which I wanted to do was to have a list of strings to store the indexing data, so that server can constantly get data and execute it.

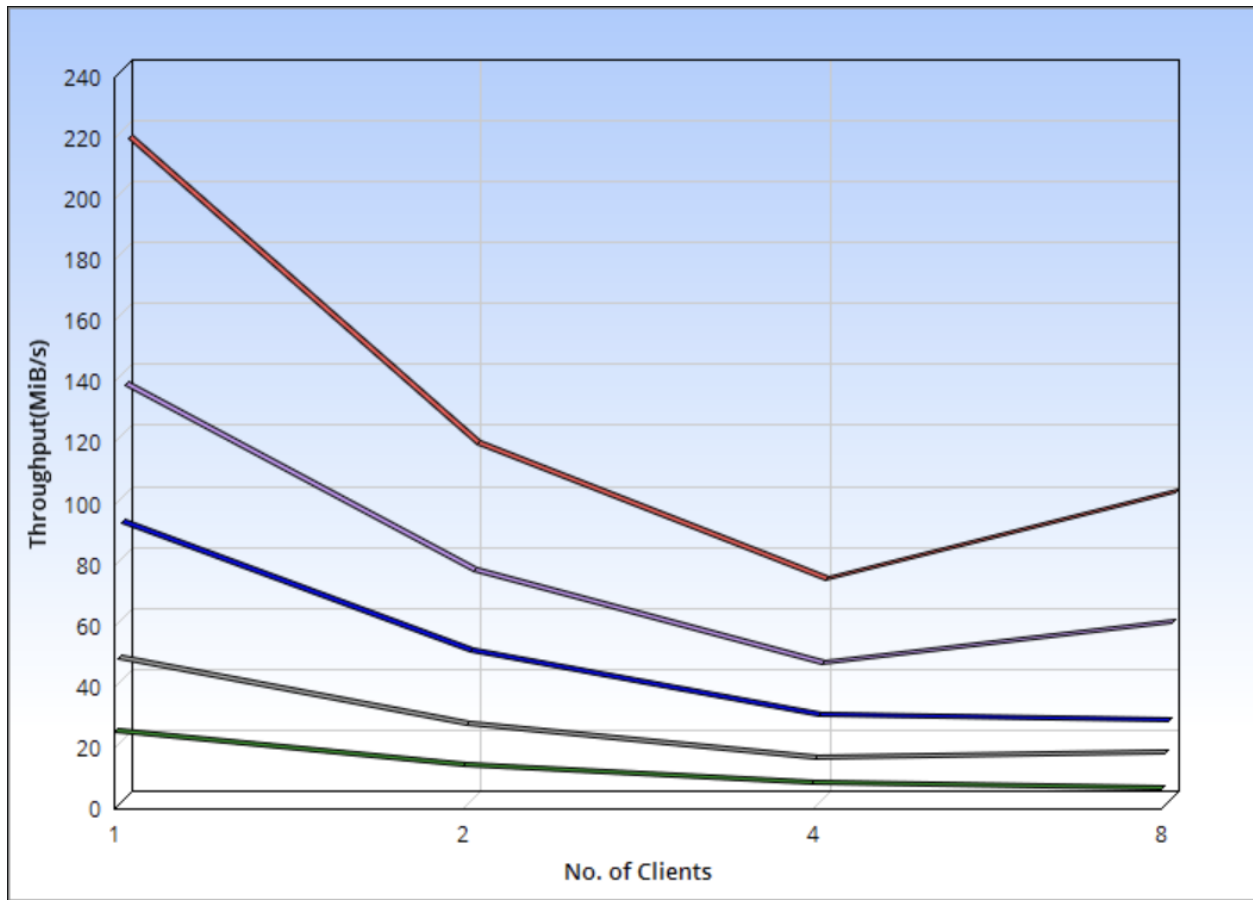
Screen snips of the plot data

Cumulative Result					
	Dataset5	Dataset4	Dataset3	Dataset2	Dataset1
1 Client	215 seconds	135 seconds	91 seconds	48 seconds	25 seconds
2 Clients	115 seconds	73.5 seconds	49 seconds	25.50 seconds	13.5 seconds
4 Clients	69.5 seconds	44 seconds	28 seconds	15.25 seconds	7.75 seconds
8 Clients	98.50 seconds	57.25 seconds	25.50 seconds	16.75 seconds	5.875 seconds
Individual Results					
2 Clients					
	Dataset5	Dataset4	Dataset3	Dataset2	Dataset1
1 Client	137 seconds	80 seconds	48 seconds	25 seconds	13 seconds
2 Clients	93 seconds	67 seconds	50 seconds	26 seconds	14 seconds
4 Clients					
	Dataset5	Dataset4	Dataset3	Dataset2	Dataset1
1 Client	72 seconds	45 seconds	28 seconds	15 seconds	7 seconds
2 Clients	91 seconds	50 seconds	27 seconds	15 seconds	8 seconds
3 Clients	56 seconds	40 seconds	27 seconds	14 seconds	7 seconds
4 Clients	59 seconds	41 seconds	30 seconds	17 seconds	9 seconds
8 Clients					
	Dataset5	Dataset4	Dataset3	Dataset2	Dataset1
1 Client	95 seconds	56 seconds	25 seconds	15 seconds	6 seconds
2 Clients	109 seconds	58 seconds	25 seconds	16 seconds	5 seconds
3 Clients	116 seconds	67 seconds	26 seconds	16 seconds	6 seconds
4 Clients	109 seconds	52 seconds	25 seconds	17 seconds	6 seconds
5 Clients	91 seconds	55 seconds	25 seconds	16 seconds	6 seconds
6 Clients	89 seconds	57 seconds	25 seconds	17 seconds	6 seconds
7 Clients	91 seconds	56 seconds	28 seconds	18 seconds	5 seconds
8 Clients	88 seconds	57 seconds	25 seconds	19 seconds	7 seconds

Continue to Next Page

CSC435 Programming Assignment - III

Screen snips of the insertion plot



Student Name: Kedar Chandulur

DePaul Student ID: 2114590

Student Email: kchandul@depaul.edu